

LAB ACTION PLAN FOR WEEK 5

<https://hub.docker.com/repository/docker/umang0007/calculator/general>

<https://hub.docker.com/repository/docker/umang0007/webapp/general>

Objectives:

Students will be able to:

- Learn how to pull, run, stop, start, remove, and inspect containers and images.
- Gain the ability to create, monitor, and troubleshoot running containers.
- Configure and manage networks for container communication.
- Create and manage persistent storage for containers.
- Learn how to list, remove, and manage images efficiently.

Docker:

Docker is an open-source platform that automates the process of building, packaging, and running applications inside lightweight, portable containers, ensuring they run consistently across different environments.

Docker Images:

A read-only template that contains the application code, runtime, libraries, and dependencies. It is like a blueprint for creating containers.

Docker Containers:

A running instance of a Docker image.

Working with Docker CLI Commands:

Docker CLI Commands with redis

Step 1: Pull the redis Image

Command:

```
docker pull redis
```

```

ASUS@Rog-Strix MINGW64 ~
$ docker pull redis
Using default tag: latest
latest: Pulling from library/redis
b1de9b9d5a81: Pull complete
db0606565b61: Pull complete
5b7942a2dab1: Pull complete
4f4fb700ef54: Pull complete
87650244ec3e: Pull complete
6310eb16bf42: Pull complete
9b30195f536d: Pull complete
bc63621f6444: Download complete
fce2fbb27b14: Download complete
Digest: sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest

```

Step 2: Run a Redis Container

Command:

```
docker run --name my-redis -d redis
```

What It Does:

- Creates and starts a container named my-redis from the redis image.
- The -d flag runs the container in the background.

```

ASUS@Rog-Strix MINGW64 ~
$ docker run --name my-redis -d redis
fb65f39d0aa49c4fed5bb8ad31575d2b0559623afb00a96f9497e95c47454b87

```

Step 3: Check Running Containers

Command:

```
docker ps
```

What It Does:

- Lists all running containers.

```

ASUS@Rog-Strix MINGW64 ~
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS          NAMES
fb65f39d0aa4   redis    "docker-entrypoint.s..." About a minute ago Up About a minute 6379/tcp       my-redis

```

Step 4: Access Redis

Command:

```
docker exec -it my-redis redis-cli
```

Opens the Redis command-line tool (redis-cli) inside the container.

Example Redis Commands:

```
127.0.0.1:6379> SET name "Alice"
```

OK

127.0.0.1:6379> GET name

"Alice"

127.0.0.1:6379>exit // to come out

```
ASUS@Rog-Strix MINGW64 ~  
$ docker exec -it my-redis redis-cli  
127.0.0.1:6379> SET name "Alice"  
OK  
127.0.0.1:6379> GET name  
"Alice"  
127.0.0.1:6379> exit
```

Step 5: Stop the Redis Container

Command:

```
docker stop my-redis
```

What It Does:

- Stops the Redis container but doesn't delete it.

```
ASUS@Rog-Strix MINGW64 ~  
$ docker stop my-redis  
my-redis
```

Step 6: Restart the Redis Container

Command:

```
docker start my-redis
```

What It Does:

- Restarts the stopped container.

```
ASUS@Rog-Strix MINGW64 ~  
$ docker start my-redis  
my-redis
```

Step 7: Remove the Redis Container

Command:

`docker rm my-redis` // Note before removing stop the container

What It Does:

- Deletes the container permanently.

```
ASUS@Rog-Strix MINGW64 ~  
$ docker stop my-redis  
my-redis  
  
ASUS@Rog-Strix MINGW64 ~  
$ docker rm my-redis  
my-redis
```

Step 8: Remove the Redis Image

Command:

`docker rmi redis`

What It Does:

- Deletes the Redis image from your local system.

```
ASUS@Rog-Strix MINGW64 ~  
$ docker rmi redis  
Untagged: redis:latest  
Deleted: sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5
```

2.Working with Docker file

A Docker file is a text file with instructions to create a custom Docker image.

Step 1: Set Up Your Folder

1. Windows:

o Create a folder like `C:\DockerProjects\Redis`.

o Open Git Bash and navigate to the folder:

`cd /c/DockerProjects/Redis`

```
ASUS@Rog-Strix MINGW64 ~  
$ mkdir -p /c/DockerProjects/Redis  
  
ASUS@Rog-Strix MINGW64 ~  
$ cd /c/DockerProjects/Redis  
  
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis  
$ |
```

2. Mac/Linux:

o Create a folder:

```
mkdir ~/DockerProjects/Redis
```

```
cd ~/DockerProjects/Redis
```

Step 2: Write the Dockerfile

1. Inside the folder, create a file named Dockerfile (no extension).

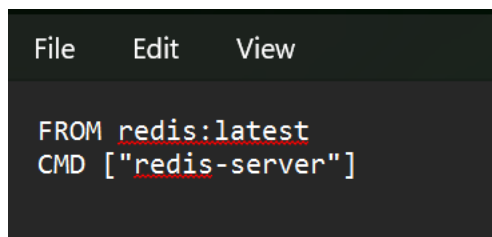
2. Add the following content:

FROM redis:latest

CMD ["redis-server"]

Docker Commands (Step-by-step):

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis  
$ notepad Dockerfile  
|
```



The screenshot shows a Notepad window with a menu bar containing 'File', 'Edit', and 'View'. The text inside the window is:

```
FROM redis:latest  
CMD ["redis-server"]
```

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis  
$ cat Dockerfile.txt  
FROM redis:latest  
CMD ["redis-server"]  
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis  
$
```

1. docker build -t redisnew .

What it does:

- This creates (builds) a Docker image using the recipe (Dockerfile) in the current folder (.).
- -t redisnew: Gives the image a name/tag ("redisnew"), so you can find it easily.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker build -t redisnew .

[+] Building 22.7s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 76B
=> [internal] load metadata for docker.io/library/redis:latest
=> [auth] library/redis:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/1] FROM docker.io/library/redis:latest@sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5
=> resolve docker.io/library/redis:latest@sha256:298e5b3bc566bade82f46ad5511777a4a07a294097ce16ada2f6a42be5239df5
=> sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 29.79MB / 29.79MB
=> sha256:87650244ec3e6580e8ea8e066d2da8bfdc7c60b92e6ee87d85c04d0a1632fcd3 25.64MB / 25.64MB
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38eacc1 32B / 32B
=> sha256:b1de9b9d5a817a02d1ecfe32acca9b8e8b7bb361185ef2d7752768b45fa075cf 2.11kB / 2.11kB
=> sha256:9b30195f536dd238e0cb439efa5f7243d14a1ef9f4c7b0308c5d726e9c3c2b36 1.10kB / 1.10kB
=> sha256:db0606565b61df7049a30a3c0b3eb6a00a903fcaf8ab62363df92ddf0c3a184 826B / 826B
=> sha256:5b7942a2dab1958d1362c600d80ea07d853522cecdcb0037a26ba8b9b58a075b 97B / 97B
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:266373b18f37ba92ce9365cca7f0ac5f70a3bbc2c852e033e54f3f30a56ca3d7
=> => exporting config sha256:a07e7a9357aef70fcb1e6687c39518a022d16ba729a8be23de697eeddf7d52db
=> => exporting attestation manifest sha256:e3f14885deef335979e74a95f5df34f5aa7865a6f2fe31aa304ae7c2517351c4
=> => exporting manifest list sha256:94da600859099b4cf31328c0615e324612ee8eb8b332da0f5d7b2157beb73799
=> => naming to docker.io/library/redisnew:latest
=> => unpacking to docker.io/library/redisnew:latest
```

2. docker run --name myredisnew -d redisnew

What it does:

- Starts a new container (mini computer) from the redisnew image.
- --name myredisnew: Names the container "myredisnew" so it's easy to identify.
- -d: Runs the container in the background.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker run --name myredisnew -d redisnew
fdee912168b64036929506f3a2bc726e392dbc482f6f547d7fc032898091d443
```

3. docker ps

What it does:

- Shows a list of containers that are running right now.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
fdee912168b6   redisnew  "docker-entrypoint.s..." 12 seconds ago Up 11 seconds 6379/tcp     myredisnew
```

4. docker stop myredisnew

What it does:

- Stops the container named "myredisnew" (like turning off a computer).

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker stop myredisnew
myredisnew
```

5. docker login

What it does:

- Logs you into your Docker Hub account, so you can upload images.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker login
Authenticating with existing credentials... [Username: umang0007]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

6. docker ps -a

What it does:

- Shows a list of all containers, including stopped ones.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fdee912168b6	redisnew	"docker-entrypoint.s..."	3 minutes ago	Exited (0) 2 minutes ago		myredisne
w1c1c6e1f2c55	ubuntu	"/bin/bash"	9 days ago	Exited (255) 52 minutes ago		img1
648fdd83f5cb	ubuntu	"/bin/bash"	9 days ago	Exited (0) 9 days ago		busy_grot
hendieck						
dab6998729de	gcc	"bash"	6 months ago	Exited (127) 6 months ago		gcc_conta
iner						
00f2e8e8426b	341bf0f3ce6c	"/docker-entrypoint.s..."	6 months ago	Exited (255) 6 months ago	0.0.0.0:8080->80/tcp	sharp_cor
i						
dfd3ed50bf1c	cd1dba651b30	"bash"	6 months ago	Exited (0) 6 months ago		admiring_
dubinsky						
7a00bcfb79da	hello-world	"/hello"	6 months ago	Exited (0) 6 months ago		bold_sato
shi						
ba5f4b98b4e0	hello-world	"/hello"	6 months ago	Exited (0) 6 months ago		youthful_
Franklin						

7. docker commit 0e993d2009a1 budarajumadhurika/redis1

// Note 0e993d2009a1 is container id of myredisnew

What it does:

- Takes a snapshot (saves changes) of the container with ID 0e993d2009a1 and creates a new image called budarajumadhurika/redis1.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker commit fdee912168b6 umang0007/redis1
sha256:34e7e62b260978c307e4f6c28060f1ae3c68603c804939febdd302a0ef0f6c36
```

8. docker images

What it does:

- Lists all images saved on your system. // will show image of budarajumadhurika/redis1

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
gcc:latest	0cc1665930da	2.19GB	569MB	U
hello-world:latest	05813aedc15f	25.9kB	9.52kB	U
redisnew:latest	94da60085909	210MB	55.4MB	U
ubuntu-git-docker:latest	0e8dee4a32d4	762MB	205MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	U
umang0007/redis1:latest	34e7e62b2609	210MB	55.4MB	
umang0007/ubuntu-git-docker:latest	0e8dee4a32d4	762MB	205MB	

9. docker push budarajumadhurika/redis1

What it does:

- Uploads the image budarajumadhurika/redis1 to Docker Hub, so others can download it.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker push umang0007/redis1
Using default tag: latest
The push refers to repository [docker.io/umang0007/redis1]
a0c845399a89: Pushed
87650244ec3e: Pushed
6310eb16bf42: Pushed
9b30195f536d: Pushed
5b7942a2dab1: Pushed
db0606565b61: Pushed
4f4fb700ef54: Pushed
b1de9b9d5a81: Pushed
latest: digest: sha256:34e7e62b260978c307e4f6c28060f1ae3c68603c804939febdd302a0ef0f6c36 size: 2035
```

10. docker rm 0e993d2009a1 // container id of myredisnew

What it does:

- Deletes the container with ID 0e993d2009a1 of myredisnew.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker rm fdee912168b6
fdee912168b6
```

11. docker rmi budarajumadhurika/redis1

What it does:

- Deletes the image budarajumadhurika/redis1 from your system.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker rmi umang0007/redis1
Untagged: umang0007/redis1:latest
Deleted: sha256:34e7e62b260978c307e4f6c28060f1ae3c68603c804939febdd302a0ef0f6c36
```

12. docker ps -a

What it does:

Shows all containers again to confirm changes.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
1c1c6e1f2c55   ubuntu        "/bin/bash"             9 days ago    Exited (255)  About an hour ago      busy_grothendieck
648fdd83f5cb   ubuntu        "/bin/bash"             9 days ago    Exited (127)  6 months ago          gcc_container
dab5998729de   gcc           "bash"                  6 months ago  Exited (255)  6 months ago          sharp_cori
00f2e8e842eb   341bf0f3ce6c  "/docker-entrypoint..." 6 months ago  Exited (0)    6 months ago          0.0.0.0:8080->80/tcp
dfd3ed50bf1c   cd1dba651b30  "bash"                  6 months ago  Exited (0)    6 months ago          admiring_dubinsky
7a00bcfb79da   hello-world    "/hello"                 6 months ago  Exited (0)    6 months ago          bold_satoshi
ba5f4b98b4e0   hello-world    "/hello"                 6 months ago  Exited (0)    6 months ago          youthful_franklin
```

13. docker logout

What it does:

- Logs you out of Docker Hub.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker logout
Removing login credentials for https://index.docker.io/v1/
```

14. docker pull budarajumadhurika/redis1

What it does:

- Downloads the image budarajumadhurika/redis1 from Docker Hub.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker pull umang0007/redis1
Using default tag: latest
latest: Pulling from umang0007/redis1
a0c845399a89: pull complete
Digest: sha256:34e7e62b260978c307e4f6c28060f1ae3c68603c804939febdd302a0ef0f6c36
Status: Downloaded newer image for umang0007/redis1:latest
docker.io/umang0007/redis1:latest
```

15. docker run --name myredis -d budarajumadhurika/redis1

What it does:

- Starts a new container using the image budarajumadhurika/redis1.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker run --name myredis -d umang0007/redis1
93f9ebaf31472ba0e69ef30b22428a4f67fdb9910375b258ce7144f57ed0fe53
```

16. docker exec -it myredis redis-cli

What it does:

- Opens the Redis command-line interface (like a terminal) inside the running container

myredis.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> |
```

17. SET name "Abcdef"

What it does:

- Saves a key-value pair in Redis (key = name, value = Abcdef).

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "Abcdef"
OK
127.0.0.1:6379>
```

18. GET name

What it does:

- Retrieves the value of the key name from Redis (it will return "Abcdef").

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "Abcdef"
OK
127.0.0.1:6379> GET name
"Abcdef"
127.0.0.1:6379>
```

19. exit

What it does:

- Exits the Redis CLI.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker exec -it myredis redis-cli
127.0.0.1:6379> SET name "Abcdef"
OK
127.0.0.1:6379> GET name
"Abcdef"
127.0.0.1:6379> exit
```

20. docker ps -a

What it does:

- Shows all containers again to check their status.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
93f9ebaf3147	umang0007/redis1	"docker-entrypoint.s..."	About a minute ago	Up About a minute	6379/tcp	myredis
1c1c6e1f2c55	ubuntu	"/bin/bash"	9 days ago	Exited (255) About an hour ago		img1
648fdd83f5cb	ubuntu	"/bin/bash"	9 days ago	Exited (0) 9 days ago		busy_grothendieck
dab6998729de	gcc	"bash"	6 months ago	Exited (127) 6 months ago		gcc_container
00f2e8e8426b	341bf0f3ce6c	"/docker-entrypoint..."	6 months ago	Exited (255) 6 months ago	0.0.0.0:8080->80/tcp	sharp_cori
dfd3ed50bf1c	cd1dba651b30	"bash"	6 months ago	Exited (0) 6 months ago		admiring_dubinsky
7a00bcfb79da	hello-world	"/hello"	6 months ago	Exited (0) 6 months ago		bold_satoshi
ba5f4b98b4e0	hello-world	"/hello"	6 months ago	Exited (0) 6 months ago		youthful_franklin

21. docker stop myredis

What it does:

- Stops the container myredis.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker stop myredis
myredis
```

22. docker rm 50a6e4a9c326

What it does:

- Deletes the container with ID 50a6e4a9c326.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker rm 93f9ebaf3147
93f9ebaf3147
```

23. docker images

What it does:

- Lists all images again to confirm which ones remain.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
gcc:latest	0cc1665930da	2.19GB	569MB	U
hello-world:latest	05813aedc15f	25.9kB	9.52kB	U
redisnew:latest	94da60085909	210MB	55.4MB	
ubuntu-git-docker:latest	0e8dee4a32d4	762MB	205MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	U
umang0007/redis1:latest	34e7e62b2609	210MB	55.4MB	
umang0007/ubuntu-git-docker:latest	0e8dee4a32d4	762MB	205MB	

24. docker rmi budarajumadhurika/redis1

What it does:

- Deletes the image budarajumadhurika/redis1 again.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker rmi umang0007/redis1
Untagged: umang0007/redis1:latest
Deleted: sha256:34e7e62b260978c307e4f6c28060f1ae3c68603c804939febdd302a0ef0f6c36
```

Step 3: Remove Login Credentials (Optional)

If you no longer need to be logged in, you can log out:

docker logout

What It Does:

- Logs you out from Docker Hub and removes your stored credentials.

```
ASUS@Rog-Strix MINGW64 /c/DockerProjects/Redis
$ docker logout
Removing login credentials for https://index.docker.io/v1/
```

Scenario based Questions:

1. Your application is running inside Docker, but you're not sure if the container is active. Which command helps you check running containers?

```
docker ps
```

2. You notice one of your containers is consuming high CPU and want to stop it. How do you stop a running container named web_app?

```
docker stop web_app
```

3. You've written a Dockerfile in your project directory and want to build an image named myapi. What Docker CLI command do you use?

```
docker build -t myapi .
```

4. You want to run your web_app container and expose its internal port 5000 on host port 8080. Which command should you use?

```
docker run -p 8080:5000 --name web_app <image_name>
```

5. Your container is running, but you want to go inside and debug it using a shell. What CLI command helps you enter the container's shell?

```
docker exec -it <container_name> /bin/bash
```

6. You've built a few test images and want to delete one called old_api. How would you do this?

```
docker rmi old_api
```

7. Your app crashed in the container. You need to check the logs. How do you check this?

```
docker logs <container_name>
```

8. You want to start a container and let it run in the background. Which command should you use?

```
docker run -d <image_name>
```

9. You're not sure which container is using port 3000. How do you check?

```
docker ps | grep 3000
```

10. You built an image but forgot to tag it. How do you tag it now?

```
docker tag <image_id> <new_image_name>:<tag>
```

11. You want to export an image to a .tar file for offline transfer. How do you export it?

```
docker save -o myimage.tar <image_name>
```

12. You want to restart a container automatically if it crashes. What Docker CLI command do you use?

```
docker run --restart always -d <image_name>
```

13. You want to see how much CPU/RAM each container uses. What Docker CLI command do you use? How to Control or Limit RAM in Docker? How to Measure Actual Memory Usage?

```
docker stats
```

```
-m or --memory
```

14. You want your container to run a shell command before starting the app. What do you include in docker file?

You use the **RUN** instruction to execute commands during the image build process (like installing dependencies), or you can use a startup bash script as your **ENTRYPOINT** to run commands right as the container boots up.

15. You want to create a custom Nginx image with your static files. What do you do?

In your project folder, create a Dockerfile that pulls the base Nginx image and copies your local HTML files into the default Nginx hosting directory. It would look like this:

```
FROM nginx:latest
```

```
COPY ./your-static-files /usr/share/nginx/html
```

Then, you build it using `docker build -t my-custom-nginx .`

Conclusion:

Through this hands-on exploration of Docker commands and Dockerfile usage, students will get a clear picture about the end-to-end container workflow—from pulling images, creating and running containers, to customizing them with Dockerfiles. By practicing key commands such as `docker build`, `docker run`, `docker ps`, and `docker exec`, students will gain practical skills in managing containerized environments. The Dockerfile exercise reinforced the importance of layered builds, efficient image creation, and environment reproducibility. Overall, these activities strengthened the understanding of containerization principles, enabling building, testing and deploying applications in isolated, consistent environments.